

Exercise #2 - Service Scheduling Feasibility Analysis

Mason McGaffin

Exercise #2

ECEN 5623 Real-Time Embedded Systems

University of Colorado Boulder

June 12, 2026

Contents

1 LINUX DEVELOPMENT SETUP	3
2 FREQUENCY EXECUTIVE ARCHITECTURE	7
2.1 BRIEF SUMMARY	7
2.2 FREQUENCY EXECUTIVE ARCHITECTURE	7
2.3 ADVANTAGES	8
2.4 DISADVANTAGES	8
3 FEASIBILITY.....	9
3.1 CHEDDAR ANALYSIS COMPARISON.....	9
3.1.0 Ex-0.....	9
3.1.1 Ex-1.....	11
3.1.2 Ex-2.....	12
3.1.3 Ex-3.....	13
3.1.4 Ex-4.....	14
3.2 IMPLEMENTING MORE EXAMPLES	16
3.2.1 Ex-5.....	16
3.2.2 Ex-6.....	17
3.2.3 Ex-7.....	19
3.2.4 Ex-8.....	20
3.2.5 Ex-9.....	22
3.2.6 Ex-10.....	23
3.4 AGREEABILITY	25
4 RTECS CHAPTER 3	26
4.1 RM LUB CONSTRAINTS	26
4.2 RM LUB ASSUMPTIONS.....	26
4.3 RM LUB DERIVATION TRICKINESS	27
5 REFERENCES.....	28
6 APPENDIX	29

Note. For the RTES course, I, Mason McGaffin, will be using the Jetson Orin Nano as my embedded system of choice. Although this system is over resourced for our application, it is convenient to use given my current situation.

1 Linux Development Setup

If you're using embedded Linux, make yourself an account on your R-Pi3b or 4b. For embedded Linux, we will use native development tools (rather than cross-compiling and cross-development commonly used for smaller-scale embedded systems).

a) Account setup and verification – You can use the default “pi” account and “raspberry” password, but it is more secure to make a user account. You can make a user account during install or later. To add an account later use “sudo adduser,” enter the well-known password, and enter user information as you see fit. Add your new user account as a “sudoer” using “visudo” right below root with the same privileges (if you need help with “vi”, here’s a quick reference or reference card– use arrows to position cursor, below root hit Esc, “i” for insert, type username and privileges as above, and when done, Esc, “:”, “wq”). The old unix vi editor was one of the first full-screen visual editors – it still has the advantage of being found on virtually any Unix system in existence, but is otherwise cryptic – it is still widely used in IT, by developers and systems engineers, so it’s good to know the basics. If you really do not like vi or vim, your next best bet is “nano,” “VS code” or “geany” for Unix systems (you can normally do “sudo apt-get install geany” or any alternative editor you like best). You are welcome to use whatever editor works best for you in this class. Finally, you’ll want an SSH tool (e.g., MobaXterm for Windows) and you’ll want to enable SSH via the GUI (or with a headless methods). Do a quick “sudo whoami” to demonstrate success for account creation.

b) Logout and test your login, then logout. Use Alt+Print-Screen to capture your desktop and save as proof you set up your account. Note that you can always get a terminal with Ctrl+Alt+t key combination. Show evidence that you have created a custom login with screenshots or photos with your smart phone. If you have any questions about your Linux systems setup, please get help from student assistants or the instructor.

c) Make sure you can access graphical tools with MobaXterm or VNC and show that tools you may need in the future work such as editors like “geany” or “eom” (sudo apt-get install geany, or sudo apt-get install eom), a tool to display graphics in PPM or PGM format. For example, show that our class web page on Firefox (or default browser) and set your home page to <https://sites.google.com/colorado.edu/ecen5623-summer>. Overall, make sure you are comfortable with development, debug, compiler general native or cross-development tools and document and demonstrate that you know them.

As mentioned in the disclaimer at the beginning of the report, I have elected to use the NVIDIA Jetson Orin Nano for this course. On the system, I have created an account user called “mason”, that has sudo group powers. I have set up a MobaXterm SSH connection to the Jetson Orin Nano device as seen in Figure 1.1. I have also established a Remote SSH connection via the VSCode Remote Explorer extension. I utilize these tools for different tasks. I prefer the command line interface of MobaXterm for direct and fast system modification. I prefer the VSCode GUI for larger file reading/writing and code development as well as the features it offers to view

10.0.0.91 (mason)

Terminal Sessions View X server Tools Games Settings Macros Help

Session Servers Tools Games Sessions View Split MultiExec Tunneling Packages Settings Help

Quick connect...

User sessions

10.0.0.91 (mason)

WSL-Ubuntu

```
mason@jetson-orin-nano:~$ groups mason
mason : mason adm cdrom sudo audio dip video plugdev render i2c lpadmin gdm gpio weston-launch sambashare
mason@jetson-orin-nano:~$
mason@jetson-orin-nano:~$ sudo whoami
root
mason@jetson-orin-nano:~$ whoami
mason
mason@jetson-orin-nano:~$
```

Jetson-orin-nano 0% 1.52 GB / 7.44 GB 0.01 Mb/s 0.02 Mb/s 35 hours mason /: 45% /boot/efi: 1%

UNREGISTERED VERSION - Please support Mobaxterm by subscribing to the professional edition here: <https://mobaxterm.mobatek.net>

The screenshot shows a VS Code editor with the file explorer on the left displaying the project structure. The main editor window shows the source code of `ex1_q4d.c`. The code includes standard headers, defines a semaphore, and implements a timer function `timer` that uses `wait` and `signal` to fire a signal after a specified time. The terminal at the bottom shows the user `mason` logging in and running `sudo whoami`, which returns `root`.

```

1  /*****
19 #define _GNU_SOURCE
20 #include <stdio.h>
21 #include <stdlib.h>
22 #include <stdint.h>
23 #include <pthread.h>
24 #include <semaphore.h>
25 #include <sched.h>
26 #include <signal.h>
27 #include <time.h>
28 #include <unistd.h>
29 #include <syslog.h>
30 #include <errno.h>
31 #include <string.h>
32 #include <sys/sysinfo.h>
33
34 #define APP_TIME_SEC 1 /* SIGALRM fires after this many seconds */
35 #define SEQ_TICK_NS (10*1000*1000ULL) /* 10 ms sequencer tick */
36 #define TICKS_PER_LCM 10 /* LCM(20ms,50ms)/10ms = 10 */
37
38 #define F10_RUNTIME_NS (10*1000*1000ULL) /* 10 ms */
39 #define F20_RUNTIME_NS (20*1000*1000ULL) /* 20 ms */

```

```

mason@jetson-orin-nano:~/RTES_5623$ whoami
mason
mason@jetson-orin-nano:~/RTES_5623$ sudo whoami
[sudo] password for mason:
root
mason@jetson-orin-nano:~/RTES_5623$

```

Figure 1.2 VSCode Remote Explorer SSH Connection

I have installed Eye of MATE Image Viewer 1.26.0 (eom) as seen in Figure 1.3. However, I prefer to use ImageMagick 6.9.11-60 as it is much faster to view images (Figure 1.4). I have elected not to install the “geany” text editor, because as I mentioned before I prefer to use the VSCode remote SSH connection for code development. I have chosen Firefox as my default browser. However, I am using X11 forwarding so it does not use the native GPU on the Jetson Orin Nano. As seen in Figure 1.5, the home page is directed to the class [homepage](#).

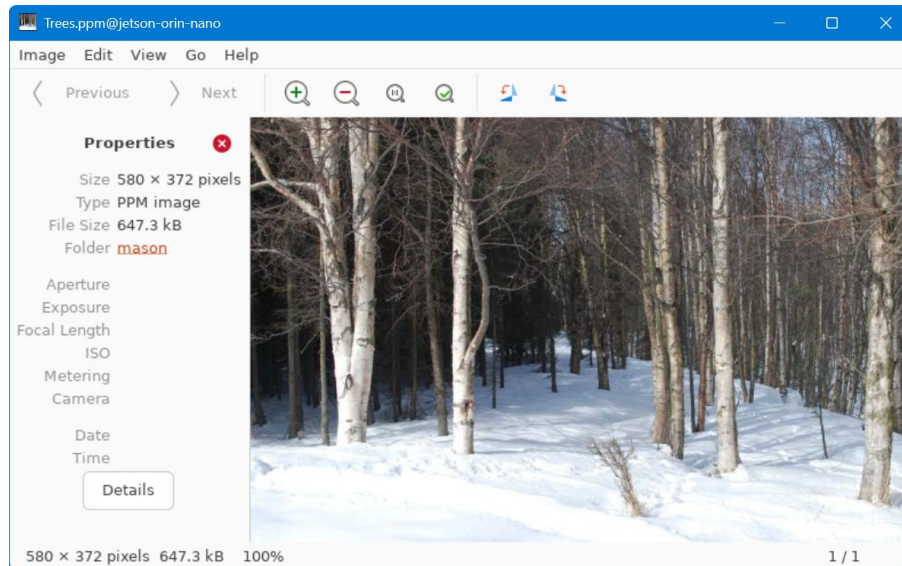


Figure 1.3 EOM graphical interface example

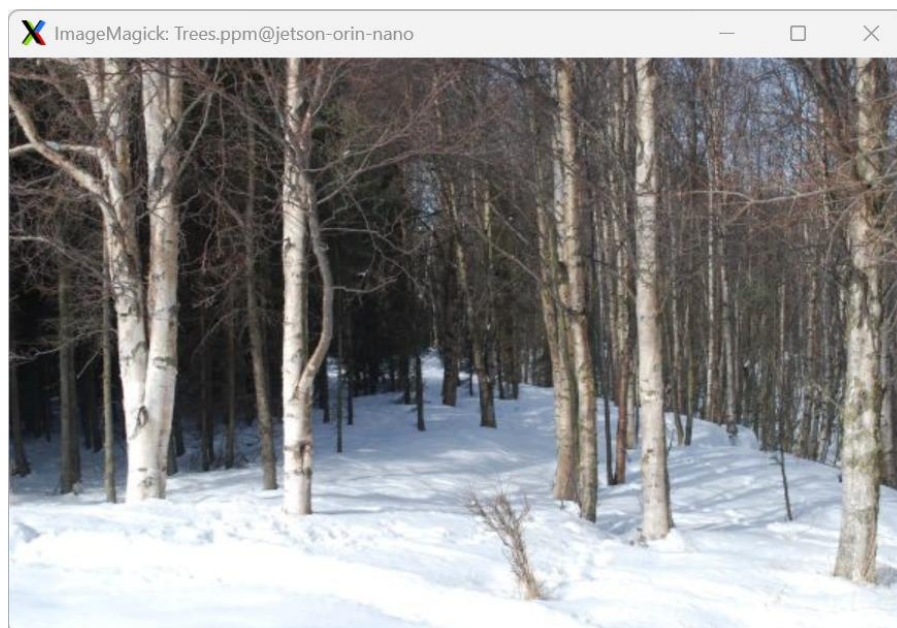


Figure 1.4 ImageMagick graphical interface example

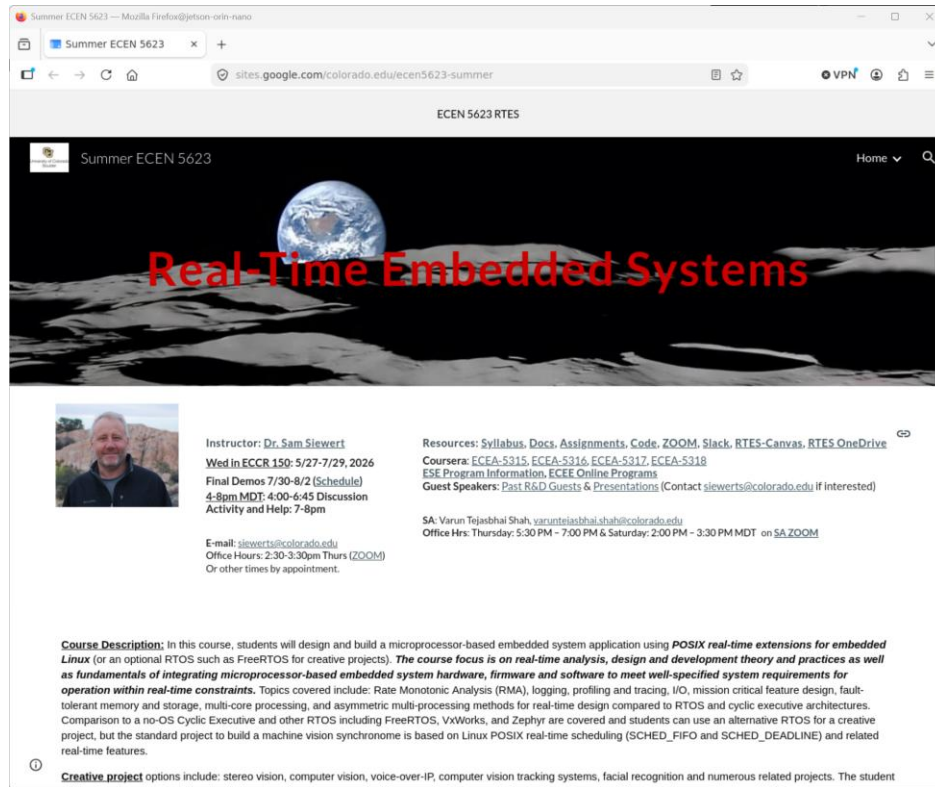


Figure 1.5 Firefox browser with class site as homepage

Lastly, I am confident with my development setup to develop, debug, and compile both natively and cross-platform. I have experience from ECEN 5713 Advanced Embedded Software Development with Linux cross-development tools and plenty of experience with gcc/g++ and Makefiles. I also am confident debugging with gdb, journalctl, strace, dmesg, and systemctl.

Build environment:

OS: Ubuntu 22.04.5 LTS
 Kernel: 5.15.148-tegra
 Architecture: aarch64
 gcc/g++ 11.4.0
 GNU Make 4.3
 GNU gdb 12.1
 GLIBC 2.35
 util-linux 2.37.2
 system 249

2 Frequency Executive Architecture

Read the paper "Architecture of the Space Shuttle Primary Avionics Software System" [available on Canvas], by Gene Carlow.

- a) Provide an explanation and critique of the frequency executive architecture.*
- b) What advantages and disadvantages does the frequency executive have compared to the real-time threading and tasking implementation methods for real-time software systems? Please be specific about the advantages and disadvantages and provide at least three advantages as well as three disadvantages.*

2.1 Brief Summary

The paper, "Architecture of the Space Shuttle Primary Avionics Software System", by Gene Carlow, describes the software architecture of the Primary Avionics Software System (PASS) on NASA's Space Shuttle program. The paper discusses the importance of a well structured system architecture over any individual tool or technique. PASS controlled nearly all shuttle operations using five (5) General Purpose Computers (GPC) and a 106k 32-bit word main memory. These extraordinary constraints were overcome by segmenting software into Operational Sequences (OPS) which are further subdivided into Major Modes, Specialist Functions (SPECs), and Display Functions (DISPs). OPS were transferred from mass memory according to the mission phase. The flight software covered three areas, Guidance, Navigation, & Control (GN&C), Systems Management (SM), and Vehicle Checkout (VCO). Within the GN&C domain, tasks are assigned priorities corresponding to their specific timing and are handled by the frequency executive architecture.

2.2 Frequency Executive Architecture

The PASS GN&C system uses a frequency executive architecture to execute functions at a fixed rate within a minor cycle. The cyclic executive is broken into three priority ordered instances. The high-frequency executive, operating at 25Hz, handles critical flight operations such as direction and thrust. The middle-frequency executive, operating at 6.25Hz, handles navigation and sensors. The low-frequency executive, operating at 0.25Hz, updates displays and background computations. The Flight Computer Operating System (FCOS) schedules uses fixed priorities according to a table-driven dispatching design to schedule each instance. The Dispatcher Table Update module (DTU) determines which functions run each cycle and allows sequence reordering based on the OPS and Major Modes.

The frequency executive architecture was a great structure for the space shuttle system given the software and hardware technologies at the time. However, as computer systems have evolved and real-time schedulers and hardware resources have improved, this architecture is best to be left to study as opposed to implementation.

2.3 Advantages

A significant advantage of using the frequency executive architecture is that it eliminates jitter from scheduler overhead. As opposed to making scheduling decisions at runtime, as done with POSIX threads, each function is called at a known time within each cycle, no matter what. This ensures exact function response timing to sensor readings.

Another advantage of this architecture is that it avoids race conditions introduced by multi-threaded processes. Instead, functions run sequentially on a single CPU. This simple design eliminates the need for locking and atomic operations and allows for deterministic execution.

The previous two reasons allow for the CPU load to be analyzed very precisely. Worst case execution times are static, and engineers can prove scheduling feasibility.

2.4 Disadvantages

Despite the advantages listed above in Section 2.3, there remain some disadvantages of the frequency executive architecture compared to real-time tasking and threading systems. First, this design eliminates concurrency. Every function is run sequentially on a single CPU, this reduces the response time of each function and the overall system speed.

The static priority scheduling has no tolerance for variable execution time. In the case a single task overruns, all subsequent tasks in the cycle are affected. Especially in space, where there is so much unknown, this design decision appears to be a large risk. Also, in the case additional function complexity is required as software evolves with missions, or in the case of reusability, it becomes very difficult to maintain stable execution times.

Another downside of this architecture is that more unused CPU cycles are wasted. As opposed to handing off the CPU to the next task, the CPU remains idle, leading to poor processor utilization. Real-time scheduling systems could achieve higher utilization on the same hardware.

3 Feasibility

Download feasibility example code and build it on your Raspberry Pi or alternate system of your choice and execute the code. Note that the C code already implements the RM LUB and RM exact feasibility analysis using scheduling point and completion test methods, but we have not studied methods to automate EDF or LLF feasibility beyond hand analysis methods, so for EDF and LLF, rely upon Cheddar and your hand analysis.

a) Compare the tests provided to analysis using Cheddar for the first 4 examples tested by the code.

b) Now, implement additional examples for practice [6 more] of your interest from those that we reviewed in class (found here). Complete analysis using Cheddar RM. In cases where RM fails (fixed priority does not work), test EDF or LLF dynamic priority policies using hand analysis or Cheddar to see if either of those succeeds and if so, explain why. Cheddar uses both service simulations over the LCM of the periods as well as feasibility analysis based on the RM LUB and scheduling-point/completion-test algorithms, referred to as “Worst Case Analysis.”

c) Does your modified feasibility code agree with Cheddar analysis for additional cases? Why or why not?

In order to compile the feasibility code without warning there was one change to be made: the previously forward slash escaped percentage sign (“... \% ...”) was replaced with two percentage signs (“... %% ...”).

3.1 Cheddar Analysis Comparison

****All units, in Section 3.1, when not specified, are assumed to be milliseconds (ms).**

Test	RM LUB	Scheduling Point	Completion Test	RM	EDF	LLF
Ex-0	F	F	F	F	F	F
Ex-1	I	I	I	I	F	F
Ex-2	I	I	I		F	I
Ex-3	I	F	F		F	F

F = FEASIBLE

I = INFEASIBLE

(X.X.X) – Reference Figure

3.1.0 Ex-0

Task	C	D=T	RM WCRT	EDF WCRT	LLF WCRT
1	1	2	1	1	1
2	1	10	2	8	8
3	2	15	6	13	13

C = Worst Case Execution Time; D = Deadline; T = Period; WRT = Worst Case Response Time

The feasibility interval is 30.

With RM, EDF, and LLF, the task set is schedulable because the processor utilization factor **0.73333** is equal or less than 1.00000 and all task deadlines will be met. See figures below.

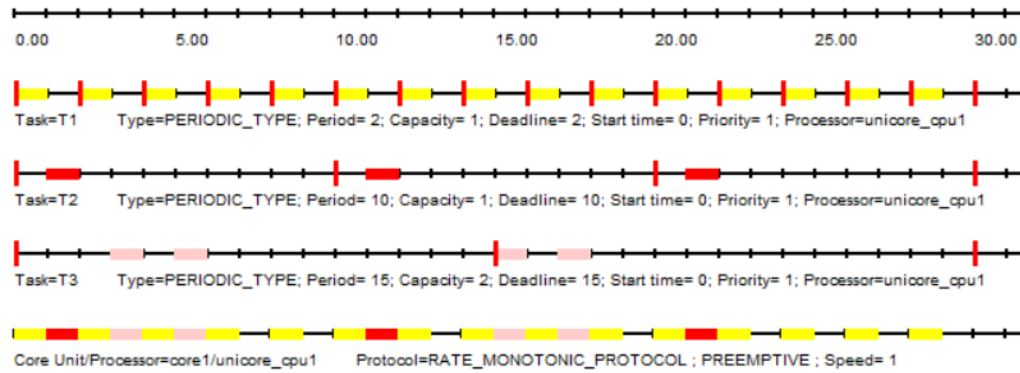


Figure 3.0.0 Test Ex-0 RM Cheddar Scheduling Simulation

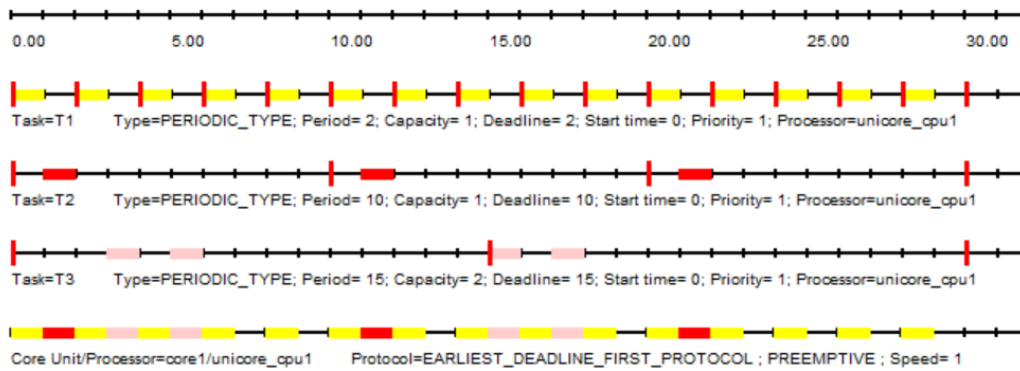


Figure 3.0.1 Test Ex-0 EDF Cheddar Scheduling Simulation

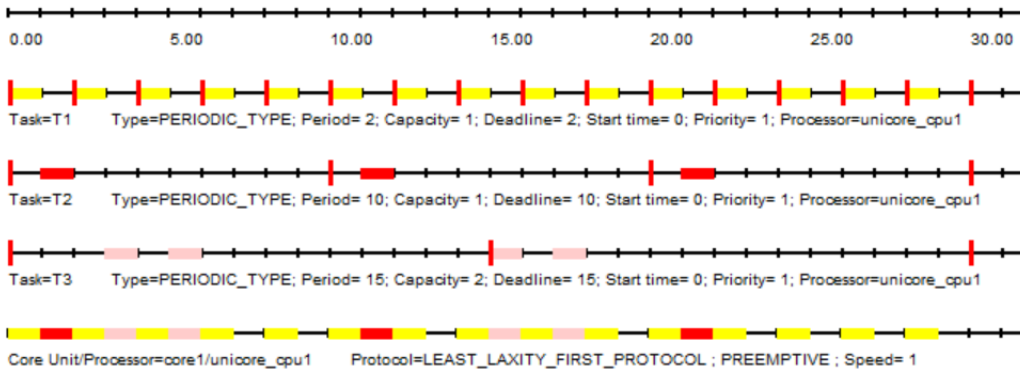


Figure 3.0.2 Test Ex-0 LLF Cheddar Scheduling Simulation

3.1.1 Ex-1

Task	C	D=T	RM WCRT	EDF WCRT	LLF WCRT
1	1	2	1	1	1
2	1	5	2	4	4
3	2	7	8 (missed)	6	6

The feasibility interval is 70.

RM feasibility analysis and simulation, Figure 3.1.0, show that the task set is not schedulable with Rate Monotonic scheduling as the first deadline is missed.

With EDF and LLF, the task set is schedulable because the processor utilization factor **0.98571** is equal or less than 1.00000 and all task deadlines will be met. See Figure 3.1.1 and Figure 3.1.2.

Worst case task response time (EDF and LLF): T1 => 1 T2 => 4 T3 => 6

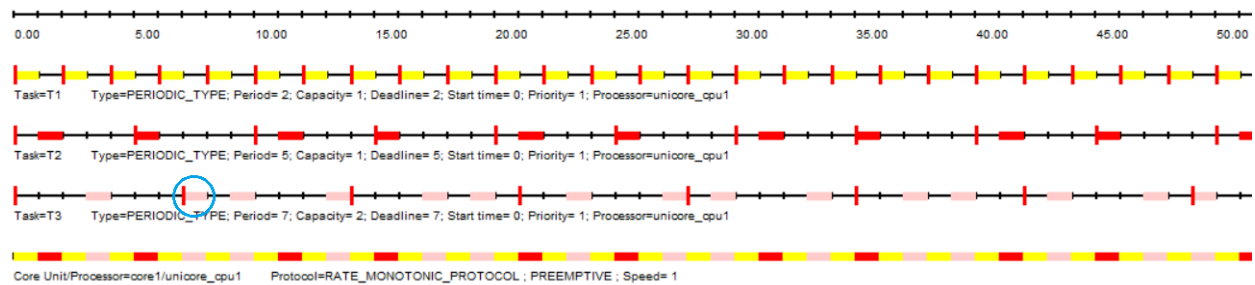


Figure 3.1.0 Test Ex-1 RM Cheddar Scheduling Simulation – First 50

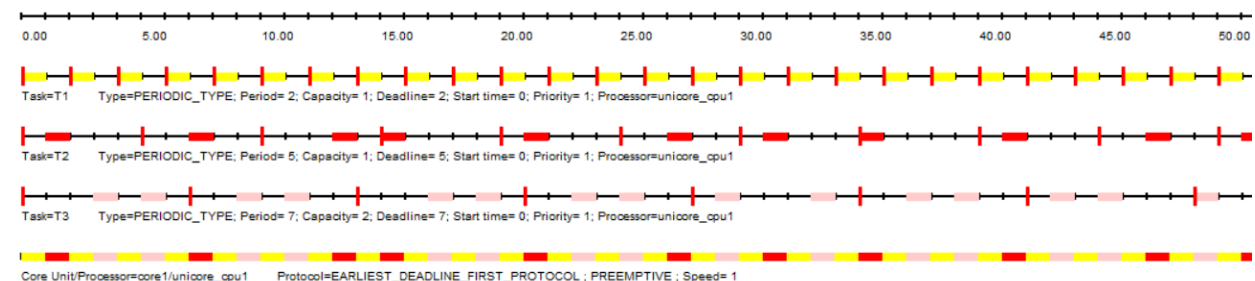


Figure 3.1.1 Test Ex-1 EDF Cheddar Scheduling Simulation – First 50

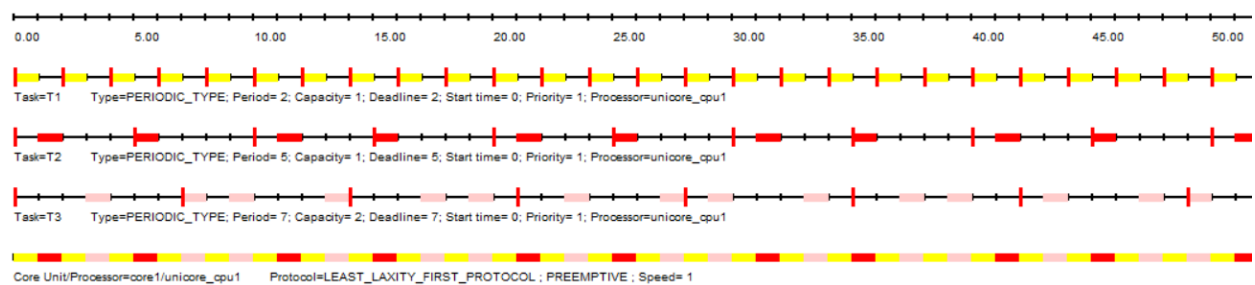


Figure 3.1.2 Test Ex-1 LLF Cheddar Scheduling Simulation – First 50

3.1.2 Ex-2

Task	C	D=T	RM WCRT	EDF WCRT	LLF WCRT
1	1	2	1	1	1
2	1	5	2	4	4
3	1	7	4	6	6
4	2	13	16 (missed)	12	16 (missed)

The feasibility interval is 910ms. The processor utilization factor **0.99670**

With RM, the task set is not schedulable because T4 misses multiple deadlines during the feasibility interval as seen in Figure 3.2.0.

With EDF, the task set is schedulable as all task deadlines are met. See Figure 3.2.1.

With LLF, the task set is not schedulable because T4 misses multiple deadlines during the feasibility interval as seen in Figure 3.2.2.

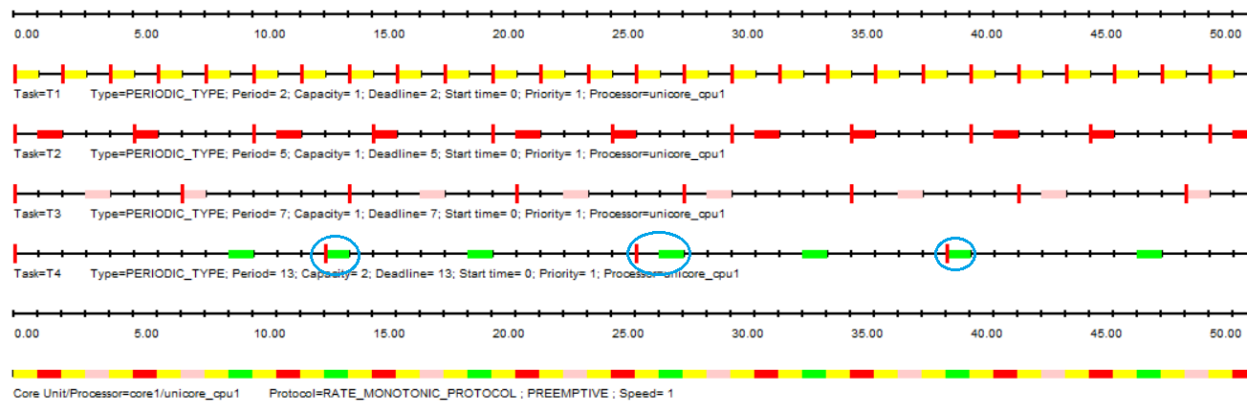


Figure 3.2.0 Test Ex-2 RM Cheddar Scheduling Simulation – First 50

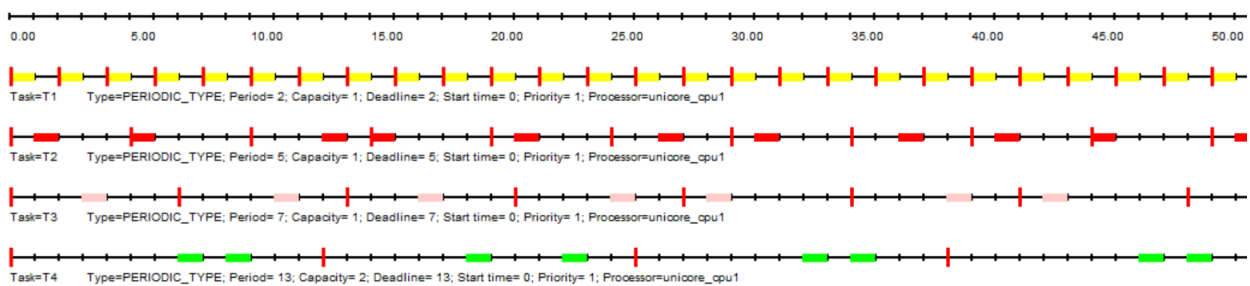


Figure 3.2.1 Test Ex-2 EDF Cheddar Scheduling Simulation – First 50

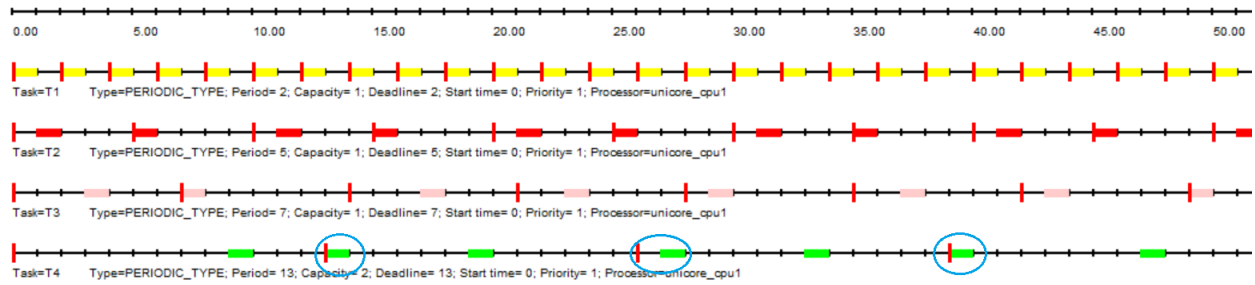


Figure 3.2.2 Test Ex-2 LLF Cheddar Scheduling Simulation – First 50

3.1.3 Ex-3

Task	C	D=T	RM WCRT	EDF WCRT	LLF WCRT
1	1	3	1	1	1
2	2	5	3	3	3
3	3	15	14	14	14

The feasibility interval is 15ms. The processor utilization factor **0.9333**

With RM, EDF and LLF, the task set is schedulable as all task deadlines are met. The scheduling protocols agree. See figures below.

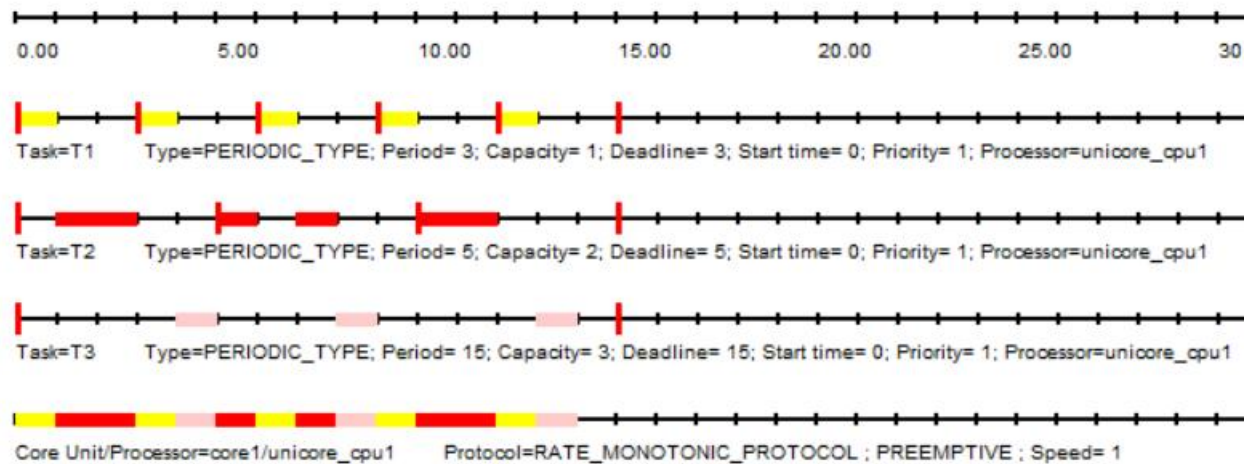


Figure 3.3.0 Test Ex-3 RM Cheddar Scheduling Simulation

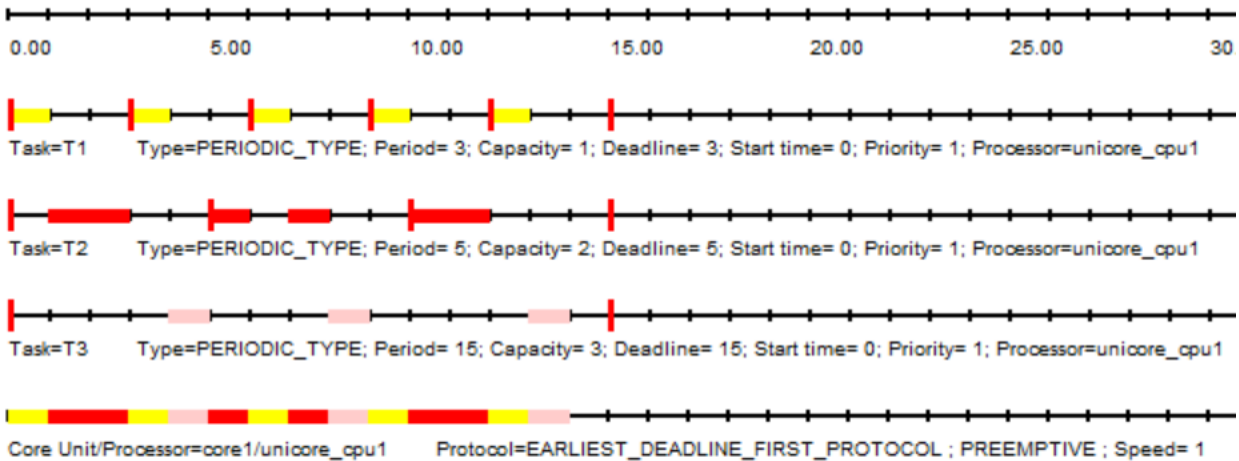


Figure 3.3.1 Test Ex-3 EDF Cheddar Scheduling Simulation

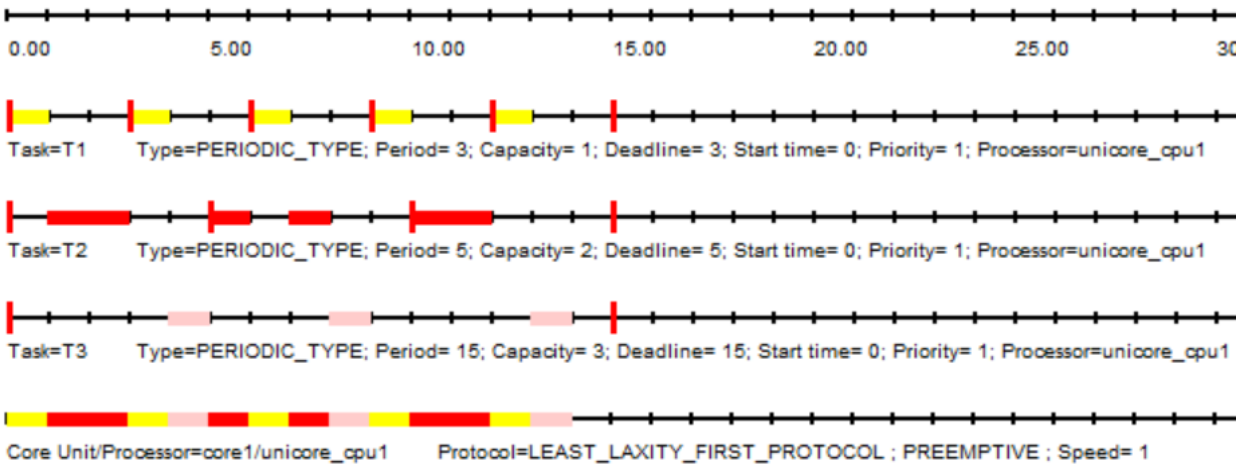


Figure 3.3.2 Test Ex-3 LLF Cheddar Scheduling Simulation

3.1.4 Ex-4

Task	C	D=T	RM WCRT	EDF WCRT	LLF WCRT
1	1	2	1	1	1
2	1	4	2	2	2
3	4	16	16	16	16

The feasibility interval is 16ms. The processor utilization factor **1.0000**

With RM, EDF, and LLF, the task set is schedulable as all task deadlines are met. The scheduling protocols agree. See figures below.

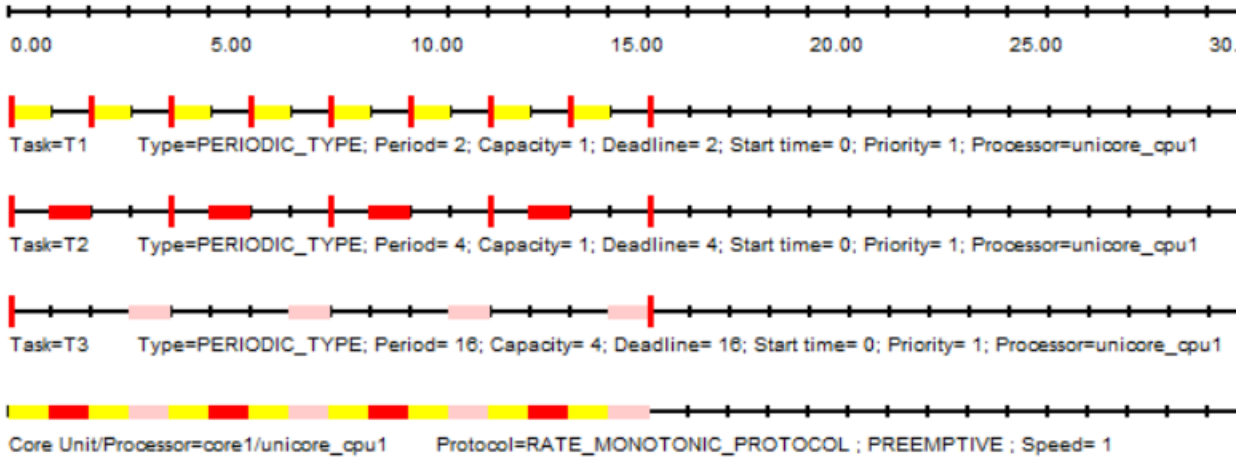


Figure 3.4.0 Test Ex-4 RM Cheddar Scheduling Simulation

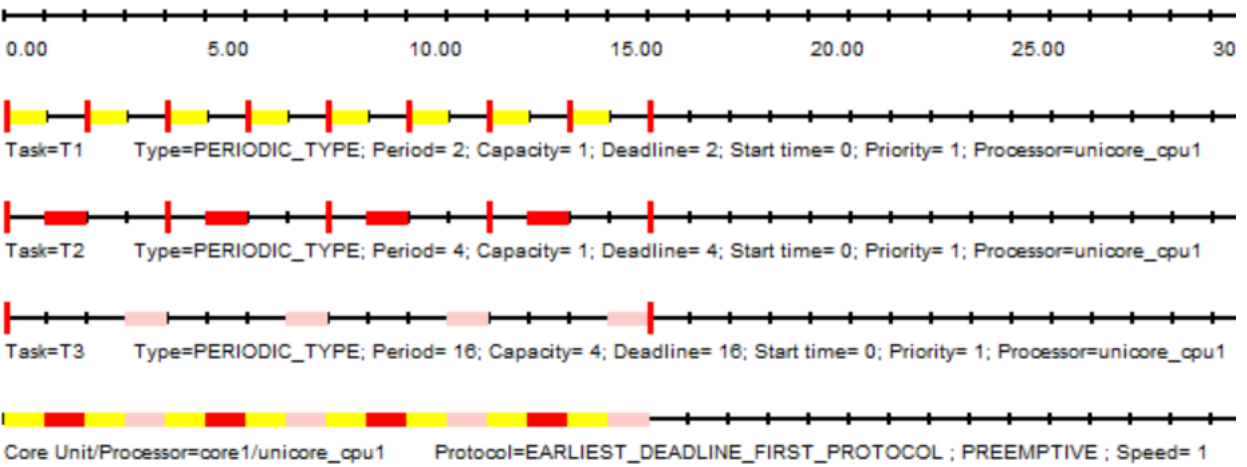


Figure 3.4.1 Test Ex-4 EDF Cheddar Scheduling Simulation

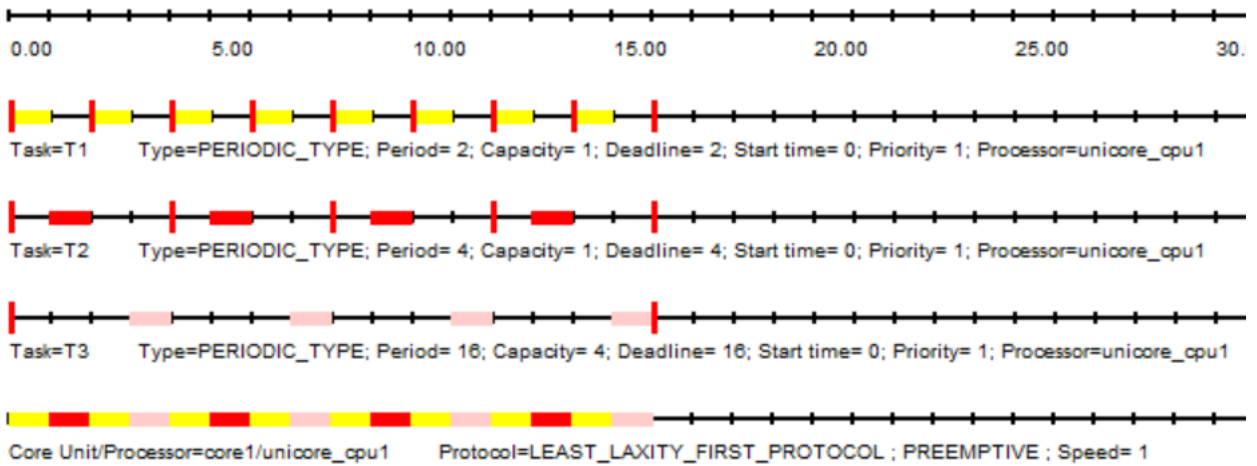


Figure 3.4.2 Test Ex-4 LLF Cheddar Scheduling Simulation

3.2 Implementing more Examples

To see modified code for implementing these example for automated RM LUB, scheduling point test, and completion test feasibility analysis, view **THIS PULL REQUEST**.

3.2.1 Ex-5

[Reference](#)

Task	C	D=T	RM WCRT	EDF WCRT	LLF WCRT
1	1	3	1	1	1
2	2	6	3	5	3
3	3	9	11 (missed)	9	11 (missed)

U=1.0000

Feasibility Interval = 9

Test	F	I	Context Switches	Preemptions
RM LUB		X		
Scheduling Point		X		
Completion Test		X	Context Switches	Preemptions
RM		X	11	2
EDF	X		13	3
LLF		X	11	2

EDF works in this case, even when RM fails, because it dynamically schedules tasks based on the nearest absolute deadline. This successfully schedules all the tasks. RM fails because priority is assigned statically upon initialization based on the length of the tasks period. This causes task 3, T3, to be interfered with by task 1, T1, and task 2, T2, and miss its deadline. EDF prevents this from happening because it allows task priorities to change as deadlines approach. LLF fails in this case because it follows the equation: $L = D - C$. Therefore, T2 has a lesser laxity ($4 = 6 - 2$) than T3 ($8 = 9 - 1$).

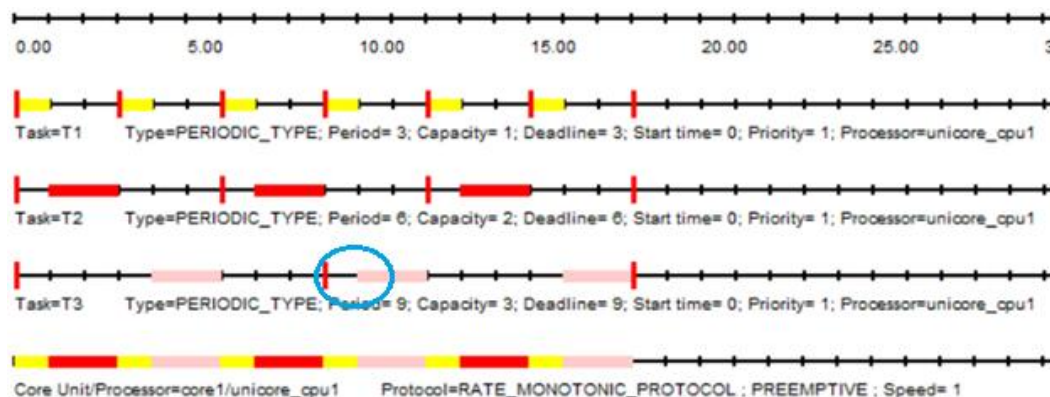


Figure 3.5.0 Test Ex-5 RM Cheddar Scheduling Simulation

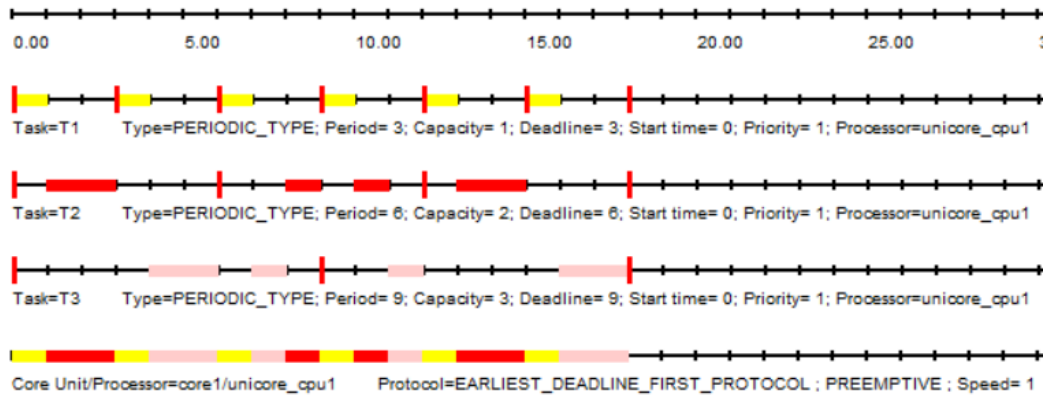


Figure 3.5.1 Test Ex-5 EDF Cheddar Scheduling Simulation

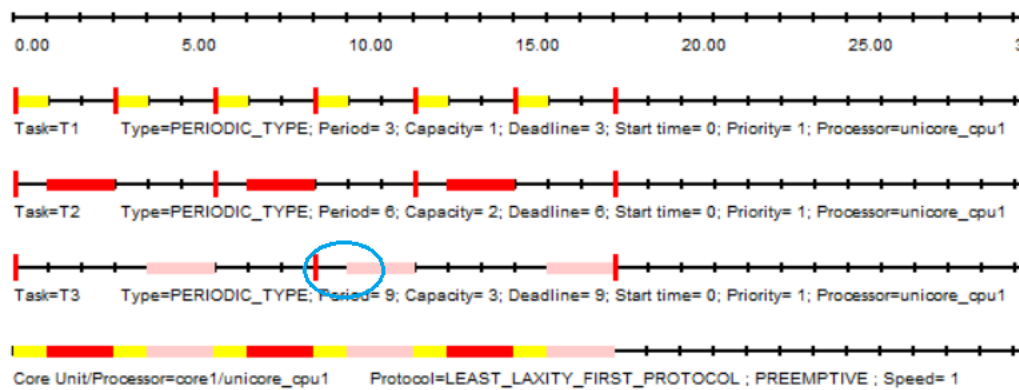


Figure 3.4.2 Test Ex-5 LLF Cheddar Scheduling Simulation

3.2.2 Ex-6

[Reference](#)

Task	C	D=T	RM WCRT	EDF WCRT	LLF WCRT
1	1	2		1	1
2	1	4		2	2
3	2	8		8	8

U=1.0000

Feasibility Interval = 8

Test	F	I		
RM LUB		X		
Scheduling Point	X			
Completion Test	X		Context Switches	Preemptions
RM	X		7	1
EDF	X		7	1
LLF	X		7	1

All feasibility tests work. Cheddar analysis shows that RM, EDF, and LLF scheduler types produce the same simulated execution schedule.

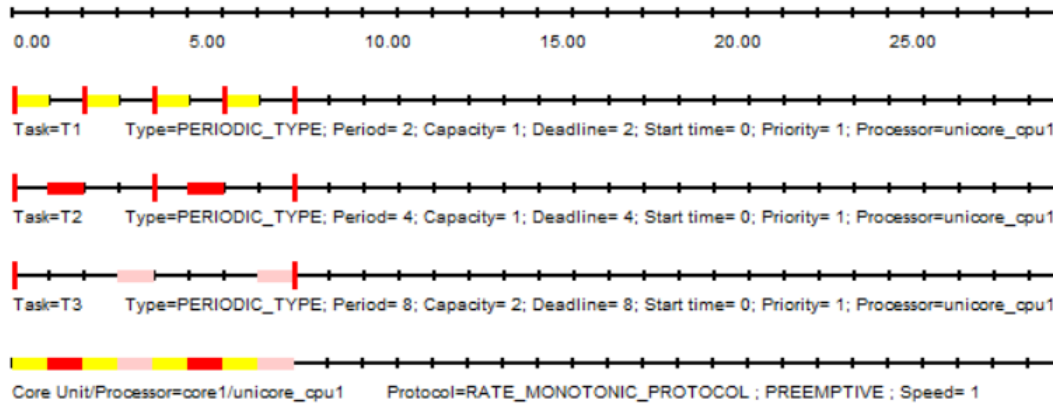


Figure 3.6.0 Test Ex-6 RM Cheddar Scheduling Simulation

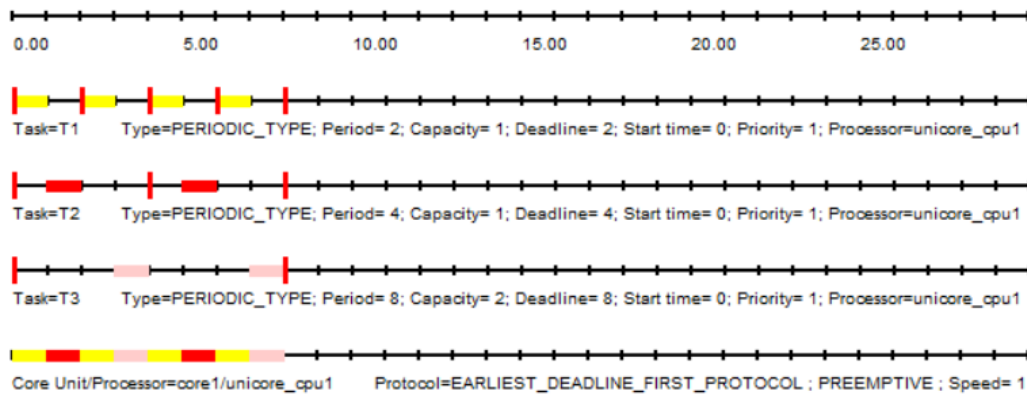


Figure 3.6.1 Test Ex-6 EDF Cheddar Scheduling Simulation

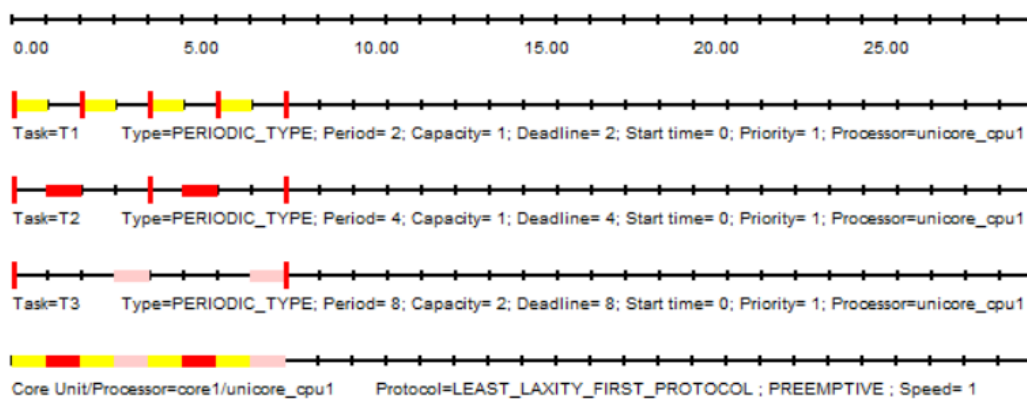


Figure 3.6.2 Test Ex-6 LLF Cheddar Scheduling Simulation

3.2.3 Ex-7

Reference

Task	C	D=T	RM WCRT	EDF WCRT	LLF WCRT
1	1	2	1	1	1
2	1	4	2	2	2
3	1	7	4	4	4

U=0.8929

Feasibility Interval = 28

Test	F	I		
RM LUB		X		
Scheduling Point	X			
Completion Test	X		Context Switches	Preemptions
RM	X		22	0
EDF	X		22	0
LLF	X		22	0

All feasibility tests work. Cheddar analysis shows that RM, EDF, and LLF scheduler types produce the same simulated execution schedule.

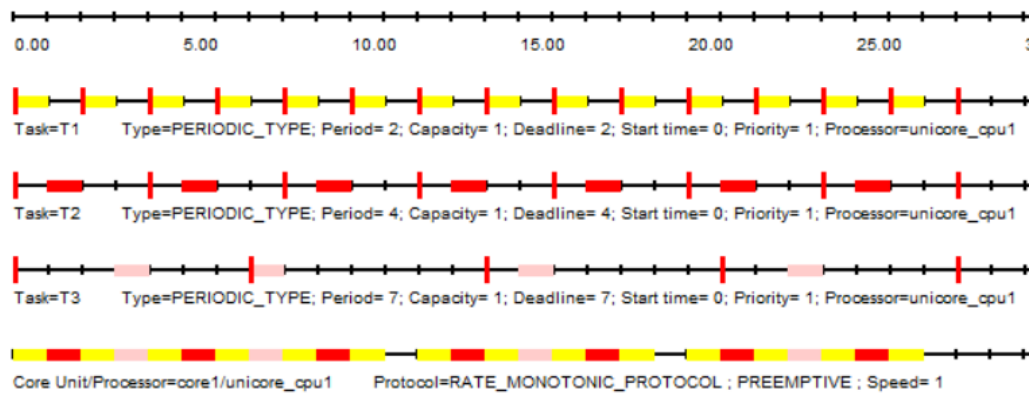


Figure 3.7.0 Test Ex-7 RM Cheddar Scheduling Simulation

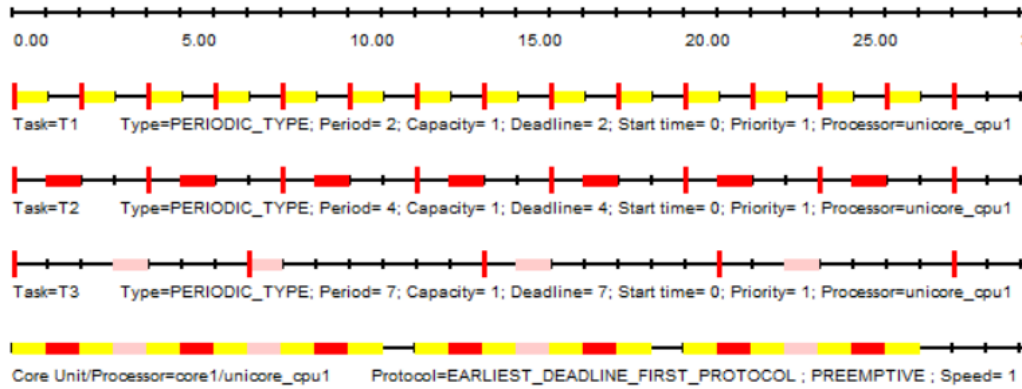


Figure 3.7.1 Test Ex-7 EDF Cheddar Scheduling Simulation

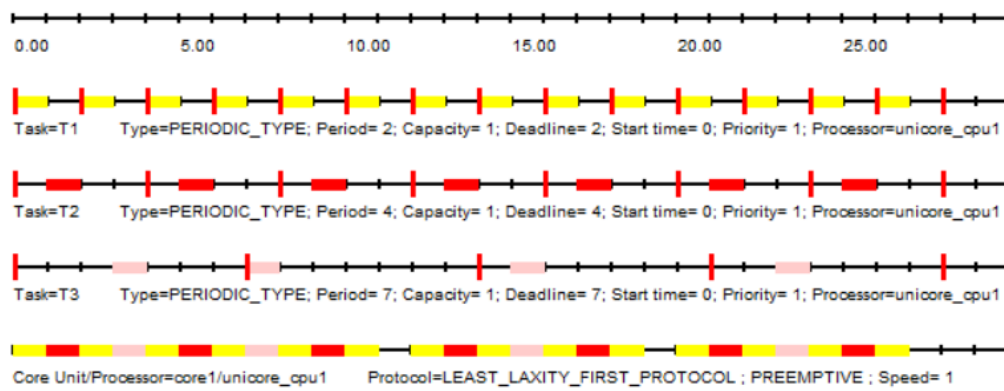


Figure 3.7.2 Test Ex-7 LLF Cheddar Scheduling Simulation

3.2.4 Ex-8

Reference

Task	C	D=T	RM WCRT	EDF WCRT	LLF WCRT
1	1	2	1	1	1
2	1	4	2	2	2
3	1	7	4	4	4
4	2	21	20	20	20

U=0.9881

Feasibility Interval = 84

Test	F	I		
RM LUB		X		
Scheduling Point	X			
Completion Test	X		Context Switches	Preemptions
RM	X		82	4
EDF	X		82	4
LLF	X		82	4

All feasibility tests work. Cheddar analysis shows that EDF and LLF scheduler types produce the same simulated execution schedule.

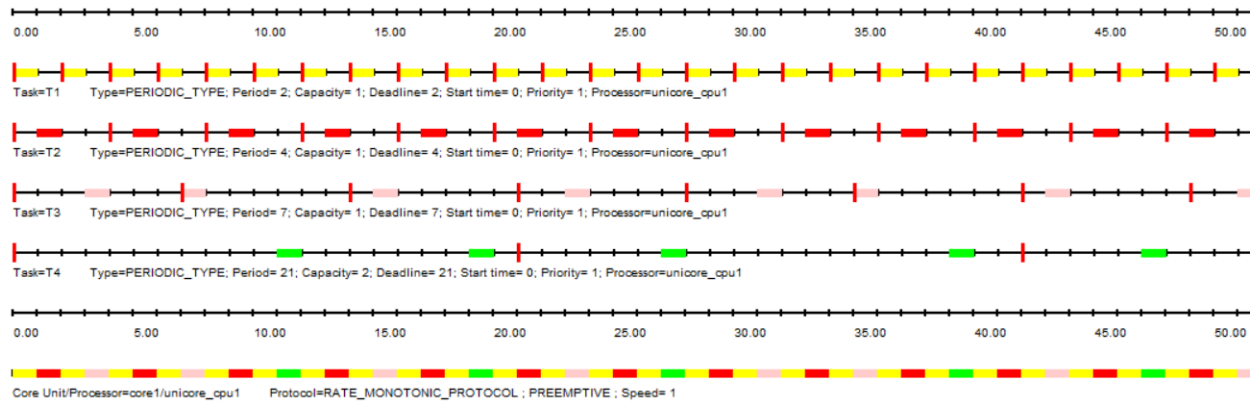


Figure 3.8.0 Test Ex-8 RM Cheddar Scheduling Simulation – First 50

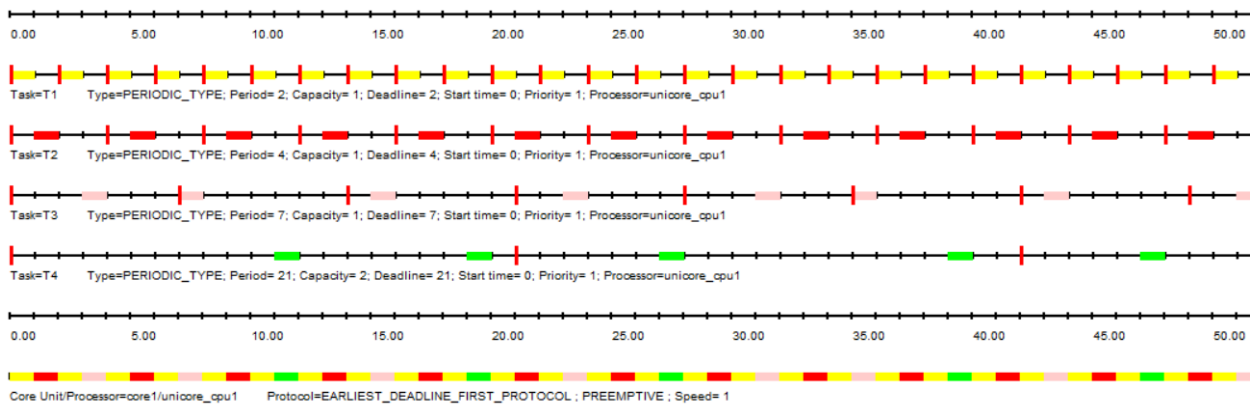


Figure 3.8.1 Test Ex-8 EDF Cheddar Scheduling Simulation – First 50

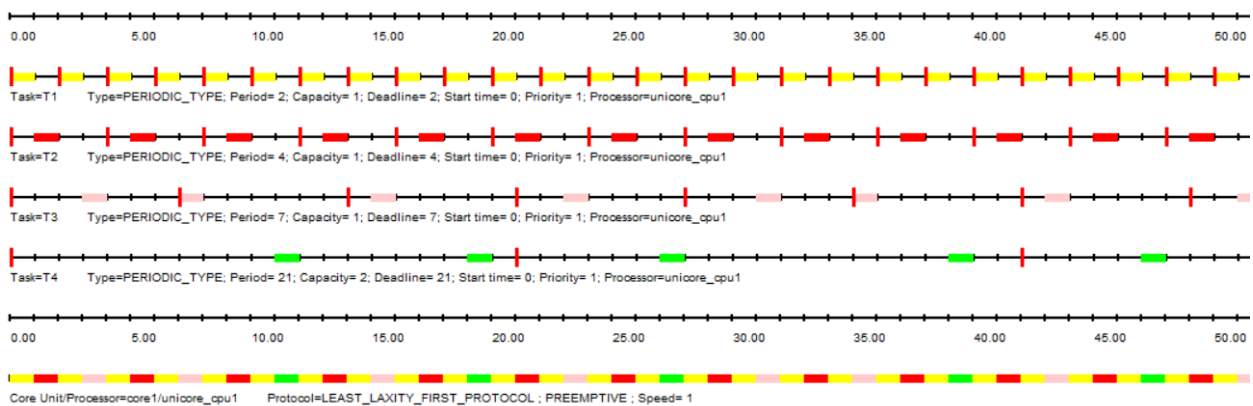


Figure 3.8.2 Test Ex-8 LLF Cheddar Scheduling Simulation – First 50

3.2.5 Ex-9

Reference

Task	C	D=T	RM WCRT	EDF WCRT	LLF WCRT
1	2	5	2	3	2
2	3	9	5	7	5
3	4	15	19 (missed)	15	19 (missed)

U=1.0000

Feasibility Interval = 45

Test	F	I		
RM LUB		X		
Scheduling Point		X		
Completion Test		X	Context Switches	Preemptions
RM		X	21	6
EDF	X		21	5
LLF		X	21	6

Similar to Ex-5, EDF works when other scheduling tests fail because it is able to dynamically change task priorities based on the nearest arriving deadline. With RM, task 3, T3, experiences significant interference from task 1, T1, and task 2, T2, because it assigned the lowest priority at the start. EDF changes the priority of T3 to accommodate for the arriving deadline. By doing this at runtime as opposed to at the start, the scheduling policy allows the task set to be feasible.

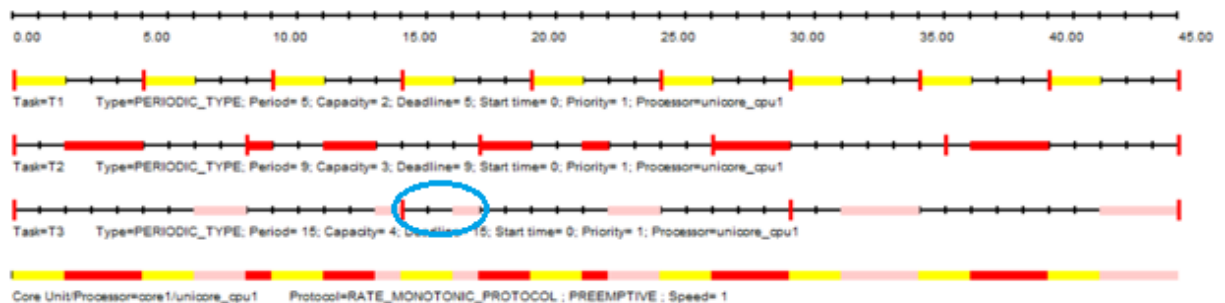


Figure 3.9.0 Test Ex-9 RM Cheddar Scheduling Simulation

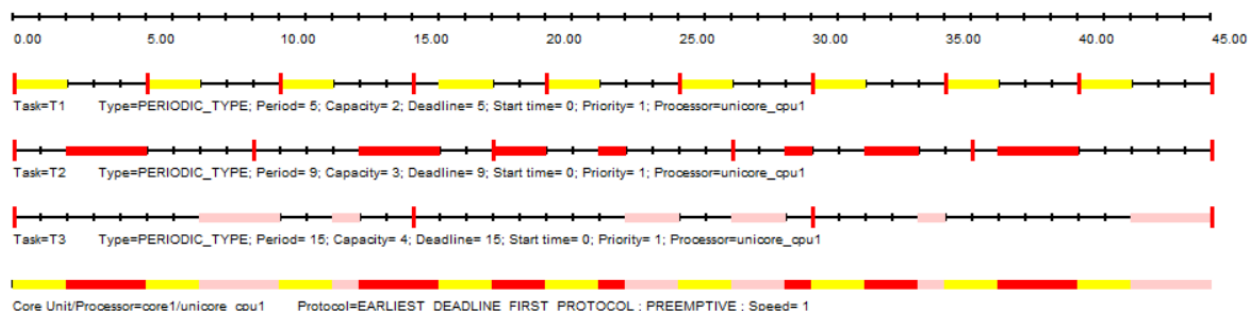


Figure 3.9.1 Test Ex-9 EDF Cheddar Scheduling Simulation

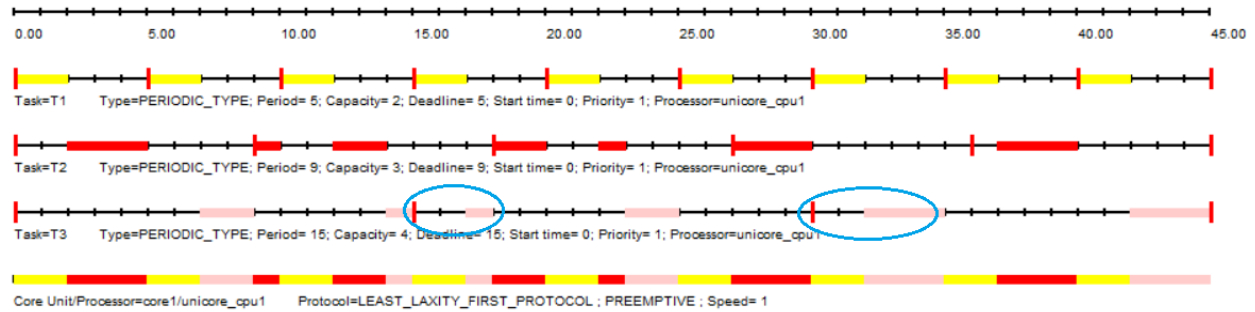


Figure 3.9.2 Test Ex-9 LLF Cheddar Scheduling Simulation

3.2.6 Ex-10

Reference

Task	C	D=T	RM WCRT	EDF WCRT	LLF WCRT
1	2	5	2	180 (missed)	2
2	2	6	4	182 (missed)	4
3	2	7	65 (missed)	181 (missed)	65 (missed)
4	2	8	0 (not run)	182 (missed)	0 (did not run)

U=1.2690

Feasibility Interval = 840

Test	F	I		
RM LUB		X		
Scheduling Point		X		
Completion Test		X	Context Switches	Preemptions
RM		X	503	84
EDF		X	417	0
LLF		X	503	84

None of the above scheduling policies can successfully schedule this task set because the processor utilization is greater than 1.000. RM fails because lower priority tasks are interfered with. Task 3, T3, misses multiple deadlines and Task 4, T4, is not scheduled because it never completes its execution. Using EDF, all tasks are attempted to be scheduled, but, because the processor is over utilized, no task completes. This is because of the excessive context switching to try to accommodate each tasks upcoming deadline. It ends up causing all tasks to miss their deadlines. LLF also fails, for a similar reason to RM. T4 is never scheduled as its execution never finishes.

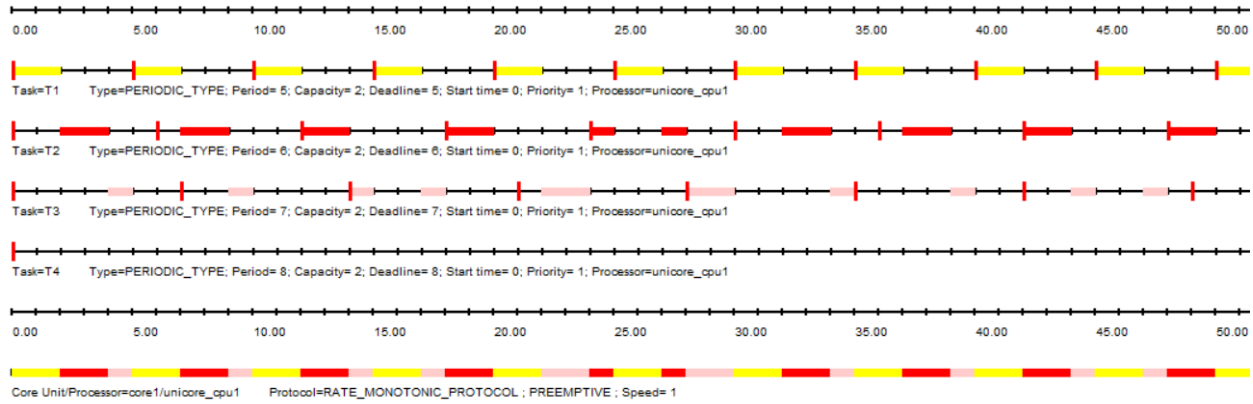


Figure 3.10.0 Test Ex-10 RM Cheddar Scheduling Simulation – First 50

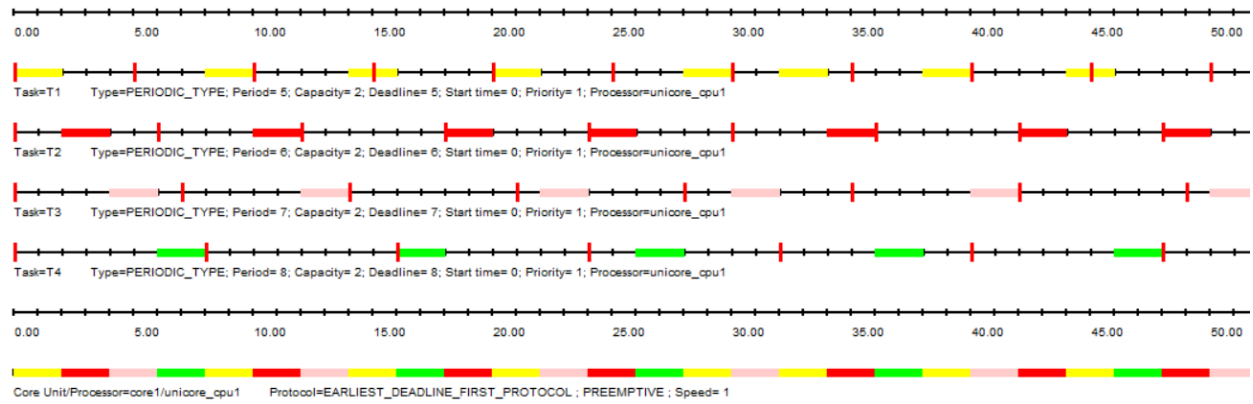


Figure 3.10.1 Test Ex-10 EDF Cheddar Scheduling Simulation – First 50

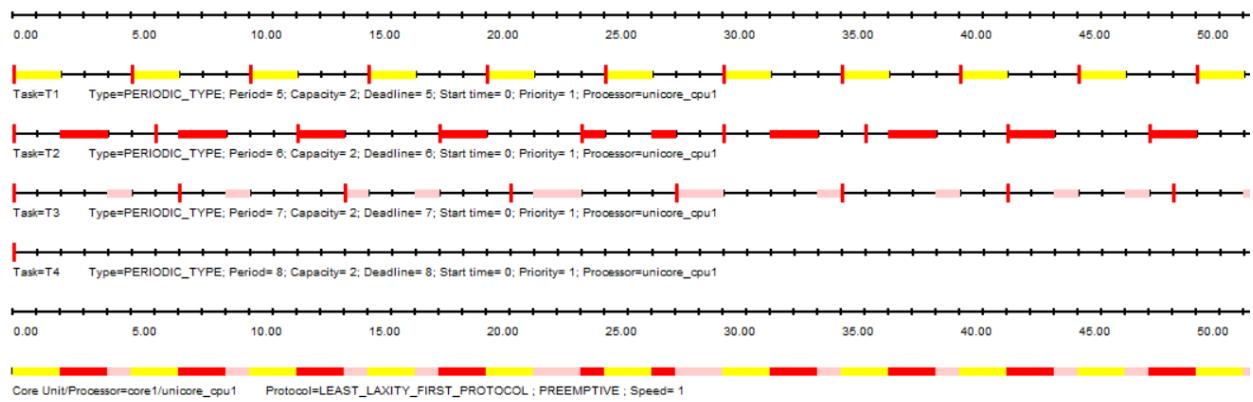


Figure 3.10.2 Test Ex-1 LLF Cheddar Scheduling Simulation – First 50

3.4 Agreeability

The feasibility code agrees with the Cheddar analysis for all six (6) examples. With RM, scheduling and worst case analysis with Cheddar consistently produced the same conclusions as the feasibility code. For examples six (6), seven (7), and eight (8), all three policies, RM, EDF, and LLF, accurately predict task set schedulability and demonstrate that all deadlines are met using the simulation. For examples five (5) and nine (9), schedulability was only feasible using EDF. Cheddar analysis confirmed the feasibility code prediction that the task sets are not schedulable with RM and also demonstrated that LLF fails. The reason EDF works is because it offers dynamic priority scheduling. LLF also uses dynamic priority scheduling, but the policy for standard LLF does not perform as well as EDF. In order to improve the LLF implementation, the scheduler could implement least runtime laxity first, which would modify the equation to $L = D - (C + t)$. This would allow the schedule to take into the account the runtime that has already passed. Example ten (10) is an overloaded example that shows that scheduling policy does not matter when the utilization of a single processor exceeds 100%. All of the analysis and simulations agreed, that the task set was not schedulable.

4 RTECS Chapter 3

Read Chapter 3 of the textbook.

a) Provide 3 constraints that are made in the RM LUB derivation and 3 assumptions as documented in the Liu and Layland paper and in Chapter 3 of RTECS with Linux and RTOS. Describe whether you think each is reasonable for actual practice or whether you think each is only applicable to an idealized model of practice.

b) Finally, list three key derivation steps in the RM LUB derivation that you either do not understand or that you would consider “tricky” math. Attempt to describe the rationale for those steps as best you can do based upon reading in Chapter 3 of RTECS with Linux and RTOS.

4.1 RM LUB Constraints

1. Fixed-priority scheduling – high-frequency tasks always have higher priority
 - a. RM policy is widely used in real-time embedded systems and is therefore reasonable for implementation. However, many modern systems use dynamic scheduling approaches such as EDF.
2. Single processor
 - a. Many embedded systems rely on a microcontroller with a single processor. It is therefore reasonable to assume this constraint is likely for practice.
3. Tasks are independent – no shared resources or synchronization.
 - a. This constraint is not as realistic. Most modern real-time embedded systems rely on mutexes, semaphores, queues, spin-locks, and shared resources. These techniques introduce blocking and priority inversion that makes the constraint not reasonable for actual practice.

4.2 RM LUB Assumptions

1. Each task has a fixed, known period, T .
 - a. Many embedded systems have periodic tasks such as sensor sampling, control loops, etc. However, these same systems also employ aperiodic or interrupt-driven functions that affect this assumption. Therefore, it is reasonable for some applications but may require additional consideration of tasks that are not periodic.
2. Each task has a deadline, D , equal to its period.
 - a. This assumption is only applicable to specific idealized practices. Especially in real-time systems, tasks deadlines often vary from the period, usually earlier. It is rarer to have a deadline after a tasks period. It is very rare for every task in a system to practice this assumption.
3. Each task has a fixed, known worst-case execution time, C .

- a. It is a goal of engineers to determine a fixed, known worst-case execution time using analysis. However, it is difficult to do as real systems are affected by caching, interrupts, and compiler optimizations. It is recommended that the WCET be assumed to be the longest possible, assuming poor performance of variable aspects. Therefore, this assumption can be reasonably applied with careful and conservative analysis.

4.3 RM LUB Derivation Trickiness

1. I was the most confused about the argument that the worst-case utilization occurs at the intersection of Case 1 and Case2. This statement initially seemed very assuming, with no great justification. However, with more careful analysis, I believe the rationale is that Case 1 utilization decreases as task 1 execution time (C_i) increases, while Case 2 utilization increases. Therefore, the intersection represents the minimum feasible utilization for both cases.
2. Another aspect that I had trouble following was the substitutions made for I and f . It was difficult to follow the substitutions that were made and the mathematical transformations that allowed the derivative and further simplification of the expression. This part was very tricky and required multiple attempts to try to make sense of the math before I was able to understand the intention and role of each step in the derivation.
3. I had a hard time following some of the 'tricky' math in the derivation, especially the decision to derive U with respect to f . This was an abrupt switch into applying calculus to the proof, but it makes sense after realizing that +we are attempting to find a minimum utilization and one way to do that is find the extrema of the derivative.

5 References

- [1] J. Pratt and S. Siewert, *Real-Time Embedded Components and Systems with Linux and RTOS*, 2nd ed. Herndon, VA, USA: Mercury Learning and Information, 2016.
- [2] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *J. ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973.
- [3] "Linux man pages," *Die.net*. Accessed: Jun. 5, 2026. [Online]. Available: <https://linux.die.net/man/>
- [4] M. Kerrisk, "Linux man-pages online," *Man7.org*. Accessed: Jun. 5, 2026. [Online]. Available: <http://man7.org/linux/man-pages/index.html>
- [5] F. Singhoff, H. N. Tran, I. Atchadam, C. Fotsing, S. Rubini, and M. Dridi, "Cheddar 3.3 User Guide," Lab-STICC Team, University of Brest, Technical Report, Jun. 26, 2023. [Online]. Available: https://beru.univ-brest.fr/cheddar/ug/ug_v3_3/
- [6] S. Siewert, "Timing Diagrams Updated 2020," *RTES-5623-CU Analysis Documents*, ESEE Web Resources, University of Colorado Boulder, 2020. [Online]. Available: https://o365coloradoedu-my.sharepoint.com/personal/siewerts_colorado_edu/_layouts/15/onedrive.aspx?id=%2Fpersonal%2Fsiewerts%5Fcolorado%5Fedu%2FDocuments%2FESEE%2DWeb%2DResources%2FRTES%2D5623%2DCU%2FDocuments%2FAnalysis%2FTiming%5FDiagrams%5FUpdated%5F2020

6 Appendix

Github repository: https://github.com/mamc3334/RTES_5623

Pull request: https://github.com/mamc3334/RTES_5623/pull/2

Forked RTES Github repository: <https://github.com/mamc3334/RTES-ECEE-5623-mcgaffin>

Pull request: <https://github.com/mamc3334/RTES-ECEE-5623-mcgaffin/pull/2>